



Sources: read these before reverse-engineering anything

This document exists because the first pass of this project derived the `song.ini` key table from YARG.Core's source when **the official wiki documents it better**, and got several things wrong as a result. The corrections are listed at the bottom. Check this page before deriving a format from source.

Primary documentation — check first

Source	Covers	Quality
YARG Wiki — <code>song.ini</code>	The full tag table with types, meanings, enum values, deprecated aliases, defaults, and a compatibility column saying whether YARG acts on a key today or merely parses it	Better than the source. The compatibility tiers and the documented defaults are not derivable from <code>SONG_INI_OUTLINES</code> at all
Guitar Game Chart Formats (TheNathannator)	<code>.chart</code> and <code>.mid</code> — both marked complete — plus <code>song.ini</code> . Audio conventions are a TODO	The reference for chart notation. The preparers in <code>internal/chart</code> were built from this , not from source
YARG Wiki — Songs	CON, ex-CON and ex-PKG layouts, <code>songs_updates</code>	Good for the RB side, which we deliberately do not support. <code>.chart</code> and <code>.mid</code> sections are stubs
YARG Wiki — <code>notes.mid</code>, <code>notes.chart</code>	Chart notation	Both are stubs; useful for YARG's recognised track list, not for note maps
mdsitton/SngFileFormat	The <code>.sng</code> container spec and the reference <code>SngCli</code> encoder	The only spec for <code>.sng</code>

Source	Covers	Quality
YARG CONTRIBUTING.md	The six-tier scope framework, including what will be rejected on sight	Authoritative on what upstream will accept

The charting documentation, found late

[Help:Charting](#) is the wiki's hub for how charts are actually authored, and it links most of what this project still needs:

Source	Why it matters here
RBN/C3 Documentation (linked from that page)	Called "the most comprehensive specification of the <code>notes.mid</code> format". Used to cross-check every note range the preparers rely on.
Chart File Format Specifications by FireFox2000000	The <code>.chart</code> spec, written by Moonscraper's author
Elite Drums MIDI/Engine Specification	A complete spec for the newest instrument, including expected game behaviour
Comprehensive song.ini Guide by grishhung	The <code>song.ini</code> reference the wiki's own table cites
opensource.yarg.in	The authoritative list of every song source YARG recognises. We treat <code>icon</code> / <code>source</code> as free text; it is an enumeration. Not yet wired in
YARN submission guidelines	Chart validity rules and, crucially, the licensing framework below

Facts from YARN that touch this code

- `icon = yarg` for YARN charts; the charting team overwrites whatever the charter set. This confirms `icon` is the *source*, which the scanner already assumes.
- `credit_license` is the **redistribution field**, not a credit line. YARN requires it on every Creative Commons and royalty-free song, formatted as `Released under CC BY-NC-SA 3.0. <link>` or `Music provided by NoCopyrightSounds. <link>`. It is now a first-class `license` field on `catalog.Song`, because a server deciding what it may serve asks a different question from a UI showing credits.
- `rating` is assigned by the charting team, not the charter — FF / SR / MC.
- Chart validity rules a preparer could check: must be tempo-mapped, at least 2 seconds of leading silence, correct track names, and **an Expert chart for every instrument charted**.
- Conventions: album art `.png` at 512×512 (500×500 accepted), backgrounds `.jpg` 16:9, and background *videos* are not allowed in YARN charts.

Phase 5 already has tools

Help:Charting names two that do the first half of LLM auto-charting, and both are free:

- **Ultimate Vocal Remover (UVR)** — separates a mix into bass, drums, vocals and other stems. That is exactly the "upload any song, get per-instrument audio" step.
- **Basic Pitch** (Spotify) — "generates pitched MIDI notes from a vocal stem", i.e. a baseline vocals chart to work from.

Phase 5 should start from these rather than from scratch.

What is genuinely NOT documented

Derived from source and from measurement, because nothing else covers it:

- **.sng has no wiki page at all.** The Songs page lists `.chart`, `.mid`, CON and ex-PKG, and does not mention `.sng`. The mdsitton spec is the only description of the container.
- **The song cache** (`songcache.bin`) — undocumented anywhere.
- **Song identity.** That a song is `SHA1(chart file bytes)` and nothing else appears in no documentation; it came from `HashWrapper` and `ScanChart`.
- **Audio stem naming** — TheNathannator lists audio as a TODO, and the clean/explicit variants are a YARG-specific extension no third-party document covers.
- **Scanner behaviour** — that a headerless `song.ini` yields no metadata, that a chart with no audio is rejected, and that a folder with no `song.ini` is silently skipped were all established by scanning a corpus with the game and reading its own report. See `TEST-CORPUS.md`.

What reading the wiki late cost

Four real defects, all shipped and then fixed:

1. `album_track` / `playlist_track` **default to 16000, not 0.** We used 0. The difference is visible to a player: unnumbered songs sort to the *front* of an album instead of the end.
2. `track` **is a deprecated alias for `album_track`, and `album_track` wins outright** when both are present. We fell back to `track` whenever `album_track` was zero — which is exactly wrong for a song explicitly numbered 0.
3. `frets` **is a deprecated alias for `charter`.** It was in our key table but mapped to nothing, so a chart using the old name lost its charter credit.
4. **Eleven keys were missing entirely** — `year_recorded`, `year_released`, `parts_vocals_harm`, `diff_guitar_coop_real`, `diff_guitar_coop_real_22`, `diff_rhythm_real`, `diff_rhythm_real_22`, `link_newgrounds`, `link_soundcloud`, `link_tiktok`.

Plus two things the wiki gives that the source cannot: the **compatibility tier** (a key can be parsed today but acted on only in some future version — presenting those as meaningful promises behaviour the client does not have), and the **meanings of the** `rating` and `vocal_gender` **enums**.

UltraStar: the format that breaks the song.ini rule

Worth stating plainly because it is a genuine exception to how every other chart works.

YARG is not MIDI-only. It reads three chart notations, chosen in this fixed priority order:

`notes.mid` → `notes.midi` → `notes.chart` → `notes.txt`. First match wins, and the selection is hard: given both a `.mid` and a `.chart`, YARG takes the `.mid` and rejects the whole song if it is unplayable, rather than falling back.

- `.chart` carries the 5-fret family, 6-fret (GHL), and 4-lane / Pro / 5-lane drums. The wiki calls it *"inferior to the notes.mid format due to its limited range of available instruments"*: vocals, harmonies, Pro Keys, Pro Guitar and Elite Drums cannot be expressed in it at all.
- `.mid` carries everything.
- `notes.txt` is UltraStar, a karaoke format, and in YARG it is **vocals-only** — no instrumental part is derived from it. `ScanUltraStar` sets lead vocals at Expert, and harmony when the chart declares `#PARTS:2`.

And UltraStar takes its metadata from the chart, not from `song.ini` — `#TITLE`, `#ARTIST` and `#ALBUM` are read from the `.txt`, and a missing or blank `#TITLE` makes YARG refuse the song with *"Name metadata not provided"*. Confirmed by measurement: a corpus case carrying a different name in each file shows the `.txt` title in YARG's cache and never the `song.ini` one.

One discrepancy noted rather than reconciled: the official UltraStar specification names voices with `#P1` ... `#P9`, but YARG keys harmony on `#PARTS`. We follow YARG, because agreeing with the client is the point, and say so where a `#P2` appears without `#PARTS:2`.

One unresolved conflict

The wiki documents `vocal_gender` as an **integer** enum (0=Female ... 4=Unspecified). YARG.Core parses the key as a **String**. Both cannot be right about the on-disk shape. The scanner stores the raw value and normalises nothing, because converting either way would be inventing a fact. Settle it by reading the modifier's conversion before anything depends on the value.